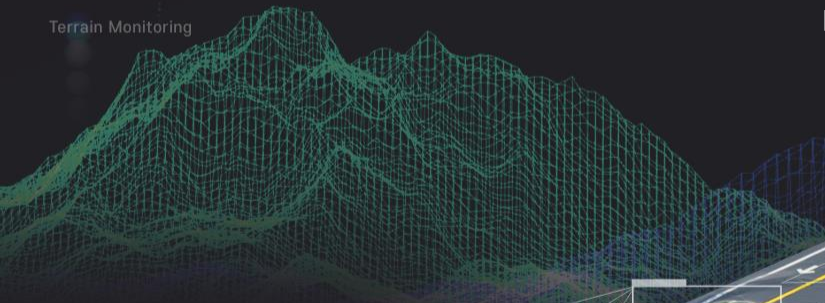


Image Recognition



Terrain Monitoring



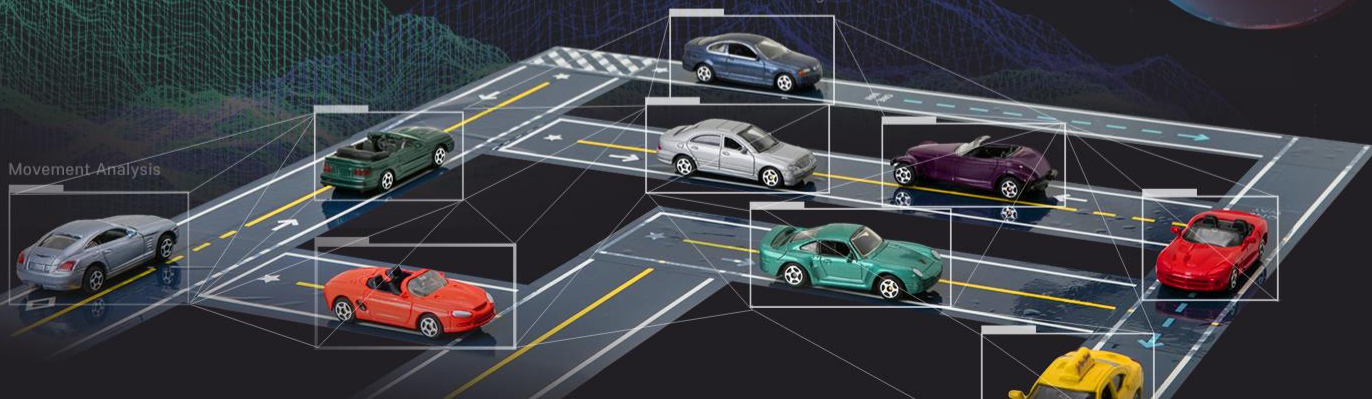
강원혁신플랫폼

컴퓨터 비전

◆
텐서플로의 이해

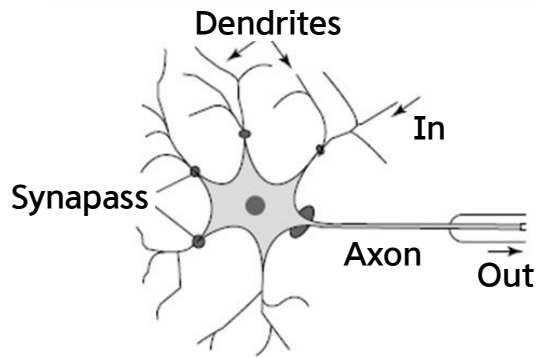
Autonomous Driving

Movement Analysis



■ 인간의 뇌를 흉내낸 인공신경망

인간의 뇌가 가지는 구조의 개략적인 특징



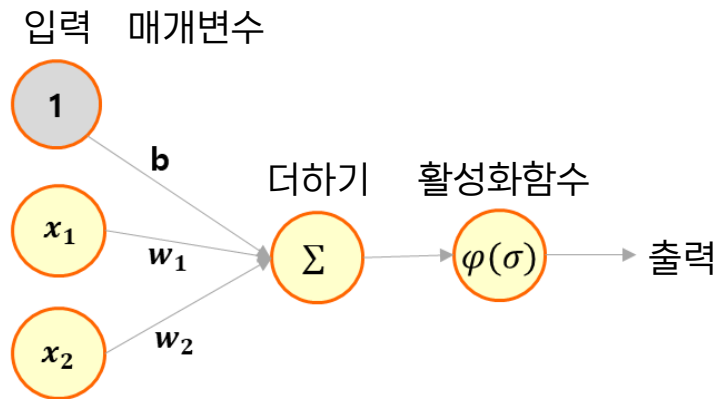
Natural neuron

자연신경망

뉴런은 수없이 많은 신경세포가
다른 신경세포와 연결되어 있음

■ 인간의 뇌를 흉내낸 인공지능망

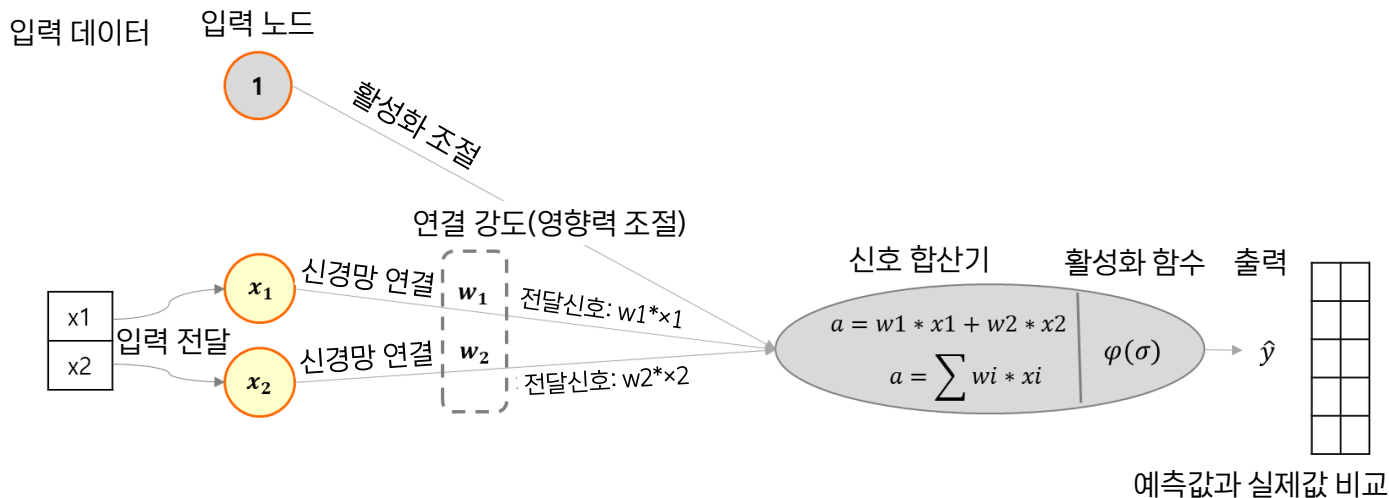
- 뉴런을 대신하는 인위적인 함수와 신경전달물질을 대신하는 숫자 값을 다른 함수의 입력으로 넘겨주는 프로그램은 충분히 구현할 수 있음
- 인공적인 신경세포(Artificial neuron)를 흉내내는 프로그램을 컴퓨터 과학자들은 퍼셉트론(Perceptron)이라고 함



신경세포의 수학적 모델(입력을 처리하여 출력으로 만드는 함수)

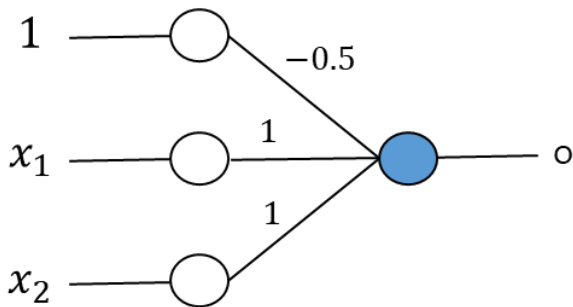
가장 단순한 인공신경망(퍼셉트론) 예시

- 그림은 두 개의 입력 노드가 하나의 출력 노드에 연결된 인공신경망을 보이고 있음
- 신경망이 동작을 변경하는 방법은 연결강도를 조정하는 것임
- 따라서, 인공신경망의 학습이라는 것은 출력을 목표치와 비교하여 오차를 계산하고 이 오차를 줄이는 방향으로 연결강도를 변경하는 일을 의미함



간단한 신경망 모델인 퍼셉트론의 OR 게이트로 텐서플로 프로그래밍을 수행해 보자.

- 좀 더 구체적으로 (1) 가중치가 정해진 학습을 마친 퍼셉트론의 동작을 확인하는 프로그램을 구현해 보자.
- 그리고 (2) OR 데이터를 학습하는 프로그램을 구현해 보자.



x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

$$y = \begin{cases} -1 & (-0.5 + w_1 x_1 + w_2 x_2) = o \leq 0 \\ 1 & (-0.5 + w_1 x_1 + w_2 x_2) = o > 0 \end{cases}$$

$$(b, w_1, w_2) \Rightarrow (-0.5, 1.0, 1.0)$$

좌측 식에 의한 OR 게이트 진리표

다음은 설정된 매개변수(가중치, 편향)로 퍼셉트론(OR 게이트)의 동작을 확인하는 코드이다.

```
import tensorflow as tf

# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[-1],[1],[1],[1]]

w = tf.Variable([[1.0],[1.0]]) # w: 가중치
b = tf.Variable(-0.5) # b: 편향
s = tf.add(tf.matmul(x,w),b) # 행렬 곱(tf.matmul)
o = tf.sign(s) # (s > 0 : 1, s <= 0 : -1)을 출력

print(o)
#tf.Tensor(
# [[-1.]
# [ 1.]
# [ 1.]
# [ 1.]], shape=(4, 1), dtype=float32)
# 실행 결과 레이블 y와 동일한 값을 출력한 것을 알 수 있다.
```

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

- 위 코드에서 매개변수 값 저장에 사용된 Variable을 np.array 배열로 바꾸어도 잘 동작함
- 텐서플로에서 Variable은 신경망의 가중치에 해당함
- 신경망을 학습하려면 기울기(그레이디언트) 정보를 구하고 가중치를 갱신하는 연산이 필요한데 이 연산을 Variable 객체가 지원함

다음은 설정된 매개변수(가중치, 편향)로 퍼셉트론(OR 게이트)의 동작을 확인하는 코드이다.

```
import tensorflow as tf

# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[-1],[1],[1],[1]]

w = tf.Variable([[1.0],[1.0]]) # w : 가중치
b = tf.Variable(-0.5)          # b : 편향
s = tf.add(tf.matmul(x,w),b)   # 행렬 곱(tf.matmul)
o = tf.sign(s)                 # (s > 0 : 1, s <= 0 : -1)을 출력

print(o)
#tf.Tensor(
# [[-1.]
# [ 1.]
# [ 1.]
# [ 1.]], shape=(4, 1), dtype=float32)
# 실행 결과 레이블 y와 동일한 값을 출력한 것을 알 수 있다.
```

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

다음은 텐서플로의 `tf.matmul()`, `tf.add()`, `tf.sign()` 함수를 사용하는 코드이다.

```
import numpy as np
import tensorflow as tf

# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[-1],[1],[1],[1]]

# OR 분류 문제의 퍼셉트론
# - w : 가중치, b : 편향
w = tf.Variable([[1.0],[1.0]])
b = tf.Variable(-0.5)

# tf.matmul() : Matrix 곱하기
# - Matrix 끼리의 곱하기를 수행하기 위해서 곱하기가 수행되는
# Matrix의 크기를 맞추주기 위해서 transpose를 해주어 연산을 해야 한다.

print(x)          # [[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]]
print(np.array(x).shape) # (4, 2)
print(w.shape)     # (2, 1)
print(tf.matmul(x,w)) # 행렬 곱(tf.matmul)
#tf.Tensor(
# [[0.]
# [1.]
# [1.]
# [2.]], shape=(4, 1), dtype=float32)
```

$$\begin{array}{|c|c|} \hline 0.0 & 0.0 \\ \hline 0.0 & 1.0 \\ \hline 1.0 & 0.0 \\ \hline 1.0 & 1.0 \\ \hline \end{array} \quad * \quad \begin{array}{|c|} \hline 1.0 \\ \hline 1.0 \\ \hline \end{array} = \begin{array}{|c|} \hline 0.0 \\ \hline 1.0 \\ \hline 1.0 \\ \hline 2.0 \\ \hline \end{array}$$

$x(4,2)$
 $w(2,1)$

다음은 텐서플로의 `tf.matmul()`, `tf.add()`, `tf.sign()` 함수를 사용하는 코드이다.

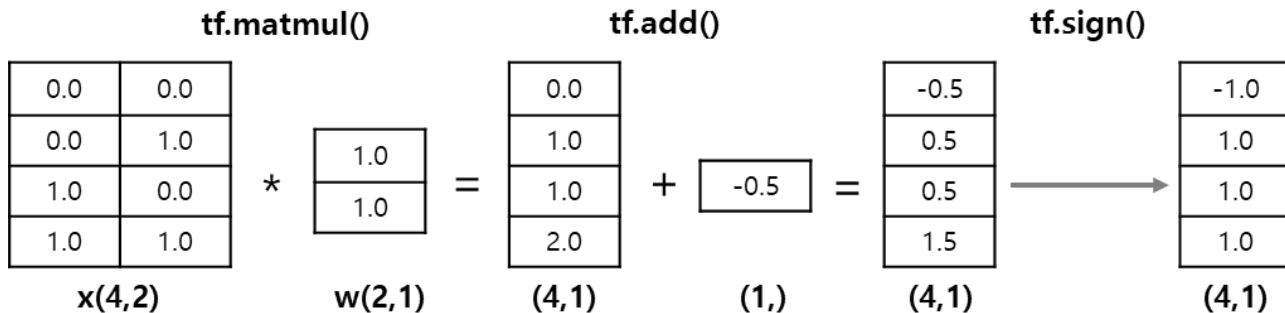
- tf.matmul() 함수 : 행렬(Matrix) 곱하기를 수행함
- 행렬끼리의 곱하기를 수행하기 위해서 곱하기가 수행되는 행렬의 크기를 맞춰주기 위해서 tranpose를 해주어 연산을 수행함
- tf.add() 함수 : 각각 원소들의 덧셈을 수행함
- tf.sign() 함수 : 부호를 리턴(음수는 -1, 양수는 1, 0은 0)함

```
print(tf.add(tf.matmul(x,w),b))    # 행렬 덧셈(tf.add)
#tf.Tensor(
# [[-0.5]
#  [ 0.5]
#  [ 0.5]
#  [ 1.5]], shape=(4, 1), dtype=float32)
```

$$\begin{array}{|c|c|} \hline 0.0 & 0.0 \\ \hline 0.0 & 1.0 \\ \hline 1.0 & 0.0 \\ \hline 1.0 & 1.0 \\ \hline \end{array} \quad x(4,2) \quad * \quad \begin{array}{|c|} \hline 1.0 \\ \hline 1.0 \\ \hline \end{array} \quad w(2,1) = \begin{array}{|c|} \hline 0.0 \\ \hline 1.0 \\ \hline 1.0 \\ \hline 2.0 \\ \hline \end{array} \quad (4,1) \quad + \quad \begin{array}{|c|} \hline -0.5 \\ \hline \end{array} \quad (1,) = \begin{array}{|c|} \hline -0.5 \\ \hline 0.5 \\ \hline 0.5 \\ \hline 1.5 \\ \hline \end{array} \quad (4,1)$$

다음은 텐서플로의 `tf.matmul()`, `tf.add()`, `tf.sign()` 함수를 사용하는 코드이다.

```
print(tf.sign(tf.add(tf.matmul(x,w),b))) # 사인 함수 값을 계산(tf.sign)
tf.Tensor(
# [[-1.]
# [ 1.]
# [ 1.]
# [ 1.]], shape=(4, 1), dtype=float32)
```



다음은 텐서플로를 이용해서 단층 퍼셉트론(OR 게이트)을 학습하는 코드이다.

```
import tensorflow as tf

# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[-1],[1],[1],[1]]

# 가중치 초기화 : w = 가중치, b = 편향
w = tf.Variable(tf.random.uniform([2,1],0.5,0.5)) # 텐서플로 학습의 경우 반드시 tf.Variable 객체 사용
b = tf.Variable(tf.zeros([1])) # 텐서플로 학습의 경우 반드시 tf.Variable 객체 사용
opt = tf.keras.optimizers.SGD(learning_rate=0.1) # 옵티마이저 : SGD(경사하강법) - 학습률 : 0.1

# 전방 계산(순전파)
def forward():
    s = tf.add(tf.matmul(x,w),b)
    o = tf.tanh(s) # 활성화 함수 : tanh
    return o

# 손실 함수 정의
def loss():
    o = forward()
    return tf.reduce_mean((y-o)**2) # (실제값-예측값)^2의 평균을 계산
```

위 코드를 보면 신경망의 원리와 수식을 이용해서 프로그램 코드를 구현한 것을 볼 수 있음

다음은 텐서플로를 이용해서 **단층 퍼셉트론**(OR 게이트)을 학습하는 코드이다.

500번 반복 학습을 수행하고 예측을 수행하는 코드

```
# 500세대까지 학습(100세대마다 학습 정보 출력)
for i in range(500):
    opt.minimize(loss, var_list=[w,b]) # opt.minimize() : 매개변수를 업데이트하는 함수
    if(i%100==0): print('loss at epoch',i,'=',loss().numpy())

# 학습된 퍼셉트론으로 OR 데이터를 예측
o = forward()
print(o)

# 실행 결과 첫 번째 샘플은 음수,
# 나머지 3개는 모두 양수로 4개 샘플을 옮겨 인식함을 확인 할 수 있다(레이블 y와 동일한 인식값 출력).
```

```
loss at epoch 0 = 1.3009424
loss at epoch 100 = 0.09975069
loss at epoch 200 = 0.044226658
loss at epoch 300 = 0.027389929
loss at epoch 400 = 0.01957656
tf.Tensor(
[[-0.8145921 ]
 [ 0.88530517]
 [ 0.88534504]
 [ 0.99924433]], shape=(4, 1), dtype=float32)
```

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

실행 결과 첫 번째 음수(=0), 나머지 3개는 모두 양수(=1)로 모두 올바르게 인식함

다음은 텐서플로를 이용해서 단층 퍼셉트론(OR 게이트)을 학습하는 코드이다.

500번 반복 학습을 수행하고 예측을 수행하는 코드

```
# 500세대까지 학습(100세대마다 학습 정보 출력)
for i in range(500):
    opt.minimize(loss, var_list=[w,b]) # opt.minimize() : 매개변수를 업데이트하는 함수
    if(i%100==0): print('loss at epoch',i,'=',loss().numpy())

# 학습된 퍼셉트론으로 OR 데이터를 예측
o = forward()
print(o)

# 실행 결과 첫 번째 샘플은 음수,
# 나머지 3개는 모두 양수로 4개 샘플을 옳게 인식함을 확인 할 수 있다(레이블 y와 동일한 인식값 출력).
```

다음은 텐서플로를 이용해서 **단층 퍼셉트론**(OR 게이트)을 학습하는 코드이다.

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

```
loss at epoch 0 = 1.3009424
loss at epoch 100 = 0.09975069
loss at epoch 200 = 0.044226658
loss at epoch 300 = 0.027389929
loss at epoch 400 = 0.01957656
tf.Tensor(
[[-0.8145921 ]
 [ 0.88530517]
 [ 0.88534504]
 [ 0.99924433]], shape=(4, 1), dtype=float32)
```

실행 결과 첫 번째 음수(=0), 나머지 3개는 모두 양수(=1)로 모두 올바르게 인식함

프로그래밍의 추상화

컴퓨터 프로그래밍은 추상화 수준을 가급적 높여 디테일을 감추는 방향으로 발전해왔음

텐서플로

엄청나게 높은 수준의 추상화를 달성한 언어

- ➡ 프로그래머는 미분을 직접 코딩하지 않으며, 옵티마이저만 지정하면 내부에서 미분을 하여 학습을 진행함
- ➡ 데이터도 샘플 하나씩 입력하지 않고 고차원 텐서로 만들어 한꺼번에 입력하면 내부에서 하나씩 학습하는 코드로 바뀌어서 실행함

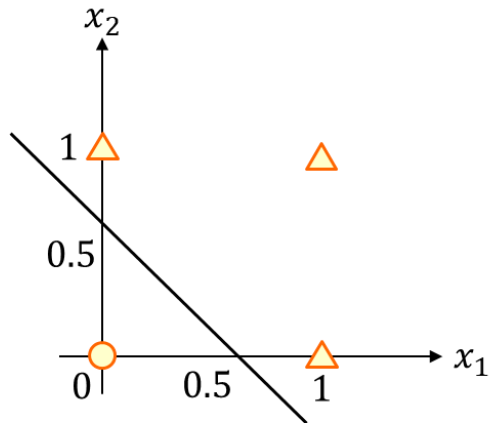
프로그래밍의 추상화

컴퓨터 프로그래밍은 추상화 수준을 가급적 높여 디테일을 감추는 방향으로 발전해왔음

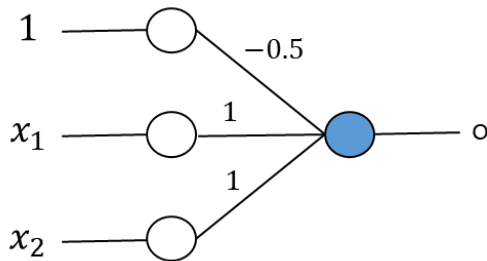
- 신경망 프로그램은 일정한 패턴으로 구성되고, 패턴을 모듈로 만들어 쌓아가는 구조이며, 같은 패턴을 함수로 만들어 디테일을 감출 여지가 남아 있음
- 정리하면, 한 단계 더 추상화함으로써 프로그래머의 수고를 크게 덜어줄 수 있으며, 케라스는 이 여지를 활용한 라이브러리임

아래 그림의 OR 게이트를 텐서플로의 케라스를 사용해 퍼셉트론을 프로그래밍해 보자.

좀 더 구체적으로 텐서플로의 케라스로 모델을 만들고 OR 데이터를 학습하여
매개변수를 찾아내는 프로그램을 구현해 보자.



OR 게이트 출력($\bigcirc=0$, $\triangle=1$)



$$y = \begin{cases} -1 & (-0.5 + w_1x_1 + w_2x_2) = o \leq 0 \\ 1 & (-0.5 + w_1x_1 + w_2x_2) = o > 0 \end{cases}$$

$$(b, w_1, w_2) \Rightarrow (-0.5, 1.0, 1.0)$$

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

■ tensorflow.keras는 Tensorflow 클래스의 하위에 Keras 클래스가 있다는 뜻임

🔄 Keras 클래스에는 Models, Layers, Optimizers 하위 클래스가 있음

Models 클래스

Sequential과
function API 모델
제작 방식 제공

Layers 클래스

다양한 종류의 층 제공

Optimizers 클래스

다양한 종류의
옵티마이저 제공

다음은 케라스를 사용한 단층 퍼셉트론(OR 게이트) 파이썬 코드이다.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

OR 데이터 구축

```
X = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
```

```
y = [[-1],[1],[1],[1]]
```

```
n_input = 2 # 2개 입력 노드 수
```

```
n_output = 1 # 1개 노드 수
```

```
perceptron = Sequential() # 신경망 구조 설계
```

```
perceptron.add(Dense(units=n_output,activation='tanh',input_shape=(n_input,),kernel_initializer='random_uniform',bias_initializer='zeros'))
```

```
perceptron.compile(loss='mse', optimizer=SGD(learning_rate=0.1), metrics=['mse']) # 신경망 학습
```

```
perceptron.fit(x, y, epochs=1000, verbose=2)
```

```
res = perceptron.predict(x) # 학습된 신경망으로 예측
```

```
print(res)
```

실행 결과 첫 번째 샘플은 음수,

나머지 3개는 모두 양수로 4개 샘플을 옳게 인식함을 확인 할 수 있다(레이블 y와 동일한 인식값 출력).

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

```
Epoch 1000/1000
```

```
1/1 - 0s - loss: 0.0069 - mse: 0.0069 - 7ms/epoch - 7ms/step
```

```
1/1 [=====] - 0s 88ms/step
```

```
[[-0.87554234]
```

```
[ 0.92240846]
```

```
[ 0.9224073 ]
```

```
[ 0.99978375]]
```

다음은 케라스를 사용한 단층 퍼셉트론(OR 게이트) 파이썬 코드이다.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

OR 데이터 구축

```
X = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
```

```
y = [[-1],[1],[1],[1]]
```

```
n_input = 2    # 2개 입력 노드 수
```

```
n_output = 1   # 1개 노드 수
```

```
perceptron = Sequential()    # 신경망 구조 설계
```

```
perceptron.add(Dense(units=n_output,activation='tanh',input_shape=(n_input,),kernel_initializer='random_uniform',bias_initializer='zeros'))
```

```
perceptron.compile(loss='mse', optimizer=SGD(learning_rate=0.1), metrics=['mse'])    # 신경망 학습
```

```
perceptron.fit(x, y, epochs=1000, verbose=2)
```

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

다음은 케라스를 사용한 단층 퍼셉트론(OR 게이트) 파이썬 코드이다.

```
res = perceptron.predict(x)    # 학습된 신경망으로 예측
print(res)
```

실행 결과 첫 번째 샘플은 음수,
나머지 3개는 모두 양수로 4개 샘플을 옳게 인식함을 확인 할 수 있다(레이블 y와 동일한 인식값 출력).

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

Epoch 1000/1000

1/1 - 0s - loss: 0.0069 - mse: 0.0069 - 7ms/epoch - 7ms/step

1/1 [=====] - 0s 88ms/step

[[-0.87554234]

[0.92240846]

[0.9224073]

[0.99978375]]

- 실행결과 첫 번째 샘플은 음수, 나머지 3개는 모두 양수로 4개 샘플을 옳게 인식함을 확인 가능함
- 즉, 모델의 예측값과 실제값이 일치함을 알 수 있음

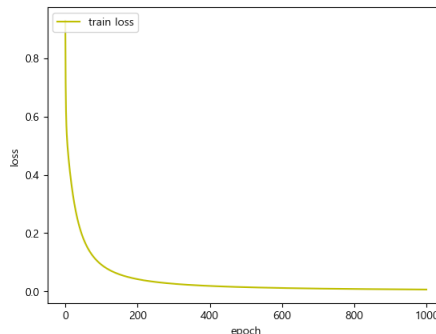
다음은 학습과정을 표시한 시각화 결과이다.

```
import matplotlib.pyplot as plt

fig, loss_ax = plt.subplots()

loss_ax.plot(hist.history['loss'], 'y', label='train loss')
loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
loss_ax.legend(loc='upper left')
plt.show()
```

- 아래 시각화 결과에서 학습을 진행하면서 손실값이 점차 감소하고 있는 것을 확인할 수 있음
- 즉, 학습이 잘 진행되고 있는 것을 알 수 있음



다음은 학습과정을 표시한 시각화 결과이다.

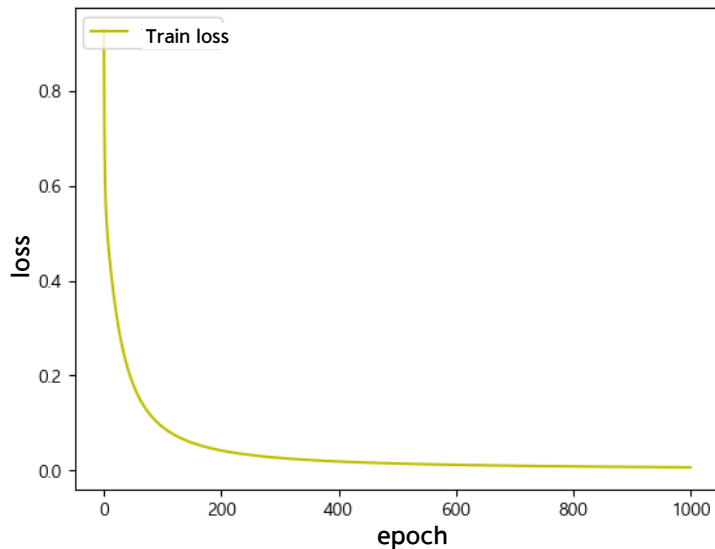
```
import matplotlib.pyplot as plt

fig, loss_ax = plt.subplots()

loss_ax.plot(hist.history['loss'], 'y', label='train loss')
loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
loss_ax.legend(loc='upper left')
plt.show()
```

다음은 학습과정을 표시한 시각화 결과이다.

- 아래 시각화 결과에서 학습을 진행하면서 손실값이 점차 감소하고 있는 것을 확인할 수 있음
- 즉, 학습이 잘 진행되고 있는 것을 알 수 있음

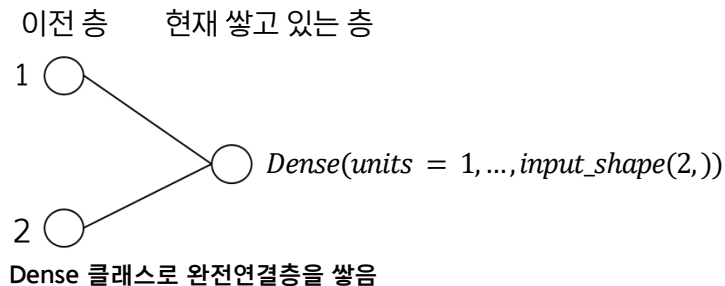


아래의 인공신경망 네트워크 구조에서 Dense 클래스로 완전연결층을 쌓은 그림은 아래 그림과 같다.

```
n_input = 2  # 2개 입력 노드 수
n_output = 1 # 1개 노드 수

# 신경망 구조 설계
perceptron = Sequential()
perceptron.add(Dense(units=n_output,activation='tanh', input_shape=(n_input,),
                      kernel_initializer='random_uniform', bias_initializer='zeros'))
```

- models 모듈의 Sequential 클래스는 층을 한 줄로 쌓는 데 사용함
- layers 모듈의 Dense 클래스는 인접한 두 층이 완전 연결된 층이 하나인 퍼셉트론 네트워크 구조를 만듦



아래의 인공신경망 네트워크 구조에서 Dense 클래스로 완전연결층을 쌓은 그림은 아래 그림과 같다.

```
n_input = 2  # 2개 입력 노드 수
n_output = 1 # 1개 노드 수

# 신경망 구조 설계
perceptron = Sequential()
perceptron.add(Dense(units=n_output,activation='tanh', input_shape=(n_input,),
                      kernel_initializer='random_uniform', bias_initializer='zeros'))
```

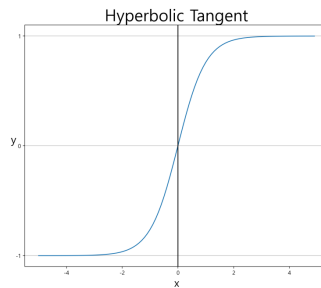
- models 모듈의 Sequential 클래스는 층을 한 줄로 쌓는 데 사용함
- layers 모듈의 Dense 클래스는 인접한 두 층이 완전 연결된 층이 하나인 퍼셉트론 네트워크 구조를 만듦

활성화 함수로 $\tanh$ (Hyperbolic Tangent) 함수를 사용한다.

```
n_input = 2  # 2개 입력 노드 수
n_output = 1 # 1개 노드 수

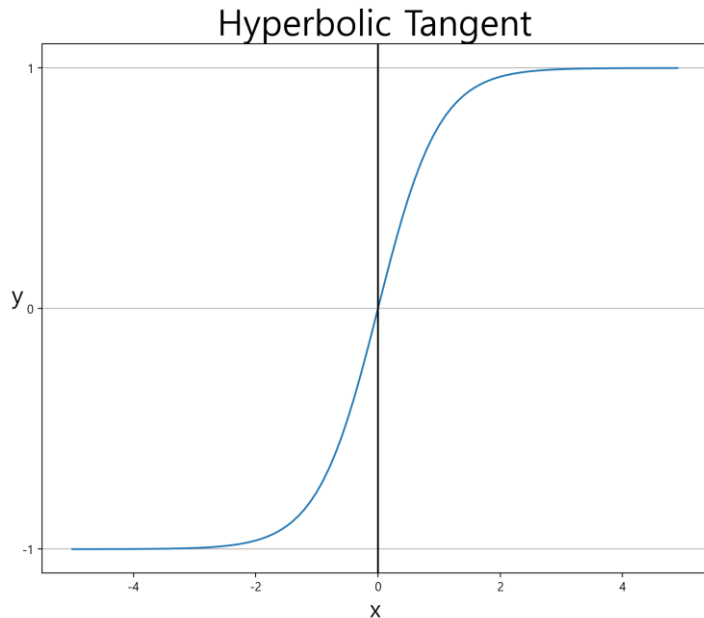
# 신경망 구조 설계
perceptron = Sequential()
perceptron.add(Dense(units=n_output, activation='tanh', input_shape=(n_input,),
                      kernel_initializer='random_uniform', bias_initializer='zeros'))
```

$\tanh$ (Hyperbolic Tangent) 함수는 좌우 대칭이라 중앙 부근이 선형인 함수로 볼 수 있음



활성화 함수로 $\tanh$ (Hyperbolic Tangent) 함수를 사용한다.

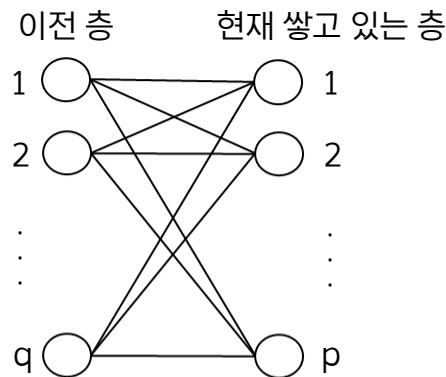
$\tanh$ (Hyperbolic Tangent) 함수는 좌우 대칭이라 중앙 부근이 선형인 함수로 볼 수 있음



다음은 케라스를 사용한 퍼셉트론 프로그래밍에서 신경망 학습 코드이다.

```
perceptron.compile(loss='mse',optimizer=SGD(learning_rate=0.1),metrics=['mse'])  
hist = perceptron.fit(x,y,epochs=1000,verbose=2)
```

- optimizers 클래스는 SGD(경사 하강법) 옵티마이저로
학습률(learning rate) 0.1로 기울기를 갱신함
- 손실 함수로 MSE(평균제곱오차)를 사용하고,
에폭(epoch)은 1000을 지정함



$Dense(units = p, \dots, input_shape(q,))$

Dense 클래스로 완전연결층을 p개 노드를 가진 층을
q개 노드를 가진 층 뒤에 쌓음

다음은 학습된 모델을 저장하고, 읽어오는 파이썬 코드이다.

```
# 학습된 모델을 h5파일로 저장한다.  
perceptron.save(os.getcwd()+'/model/single_perceptron_model.h5')  
print("Saved model to disk !!!")
```

```
import os  
from tensorflow.keras.models import load_model  
  
model = load_model(os.getcwd()+'/model/single_perceptron_model.h5')  
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1)	3
Total params: 3		
Trainable params: 3		
Non-trainable params: 0		

다음은 학습된 모델로 새로운 데이터를 예측하는 파이썬 코드이다.

```
# OR 데이터 구축
x = [[0.0,0.0],[0.0,1.0],[1.0,0.0],[1.0,1.0]]
y = [[-1],[1],[1],[1]]

# 학습된 신경망으로 예측
res = model.predict(x)
print(res)
#1/1 [=====] - 0s 38ms/step
#[[-0.87554234]
# [ 0.92240846]
# [ 0.9224073 ]
# [ 0.99978375]]
```

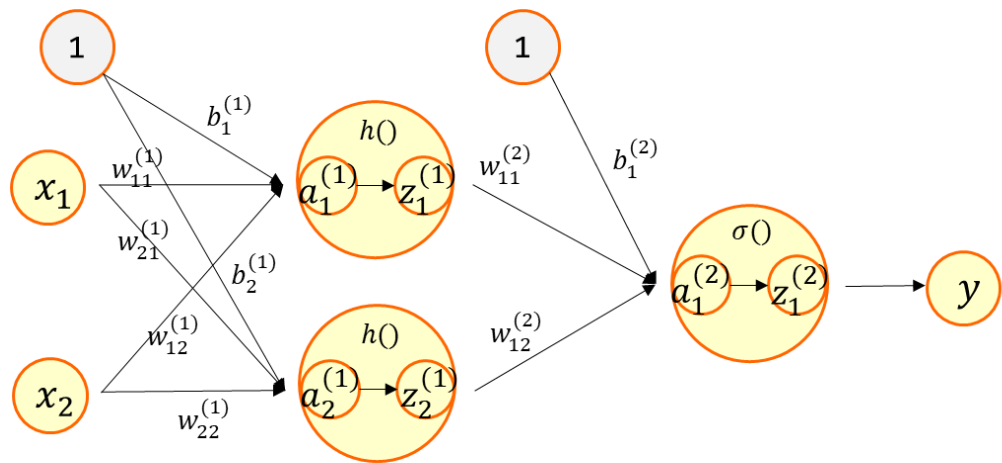
x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	1

좌측 식에 의한 OR 게이트 진리표

- 실행결과 첫 번째 샘플은 음수, 나머지 3개는 모두 양수로 4개 샘플을 옳게 인식함을 확인 할 수 있음
- 즉, 모델의 예측값과 실제값이 일치함을 알 수 있음

케라스를 이용해 XOR 게이트 학습 및 추론을 수행하는 프로그래밍해 보자.

- 아래 그림의 XOR 게이트를 텐서플로의 케라스를 사용해 퍼셉트론을 프로그래밍해 보자.
- 여기서 활성화 함수를 시그모이드(sigmoid)와 하이퍼볼릭 탄젠트(tanh) 함수를 각각 적용해보자.

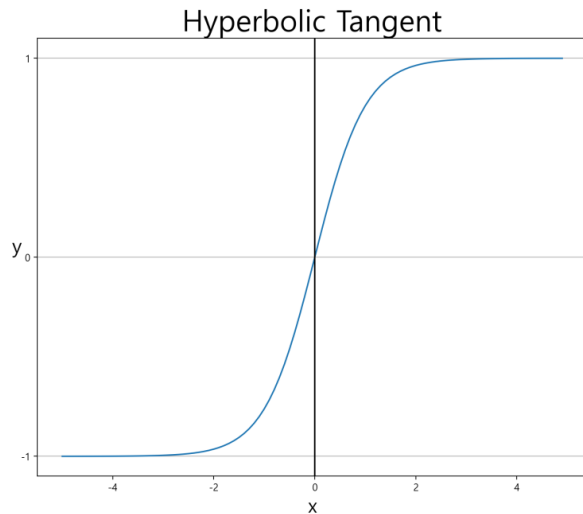


XOR 신경망

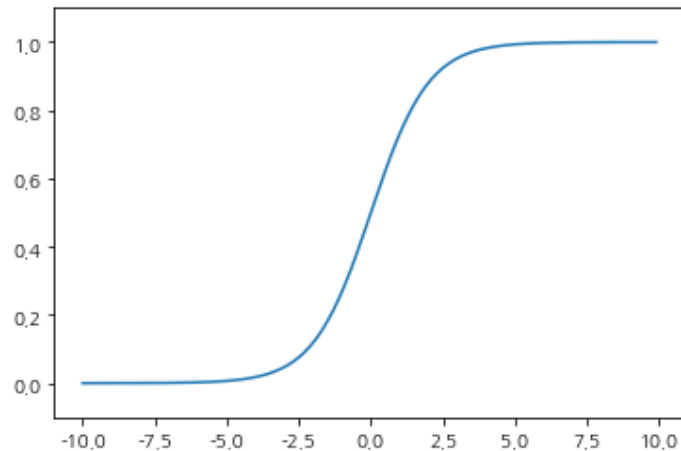
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR 게이트의 진리표

아래 그림은 활성화 함수인 하이퍼볼릭 탄젠트와 시그모이드 함수의 그래프이다.



하이퍼볼릭 탄젠트(Tanh)



시그모이드(Sigmoid)